

examples are shown below:

```
(-some predicate list) ;; Syntax
```

```
(-some 'even? '(1 2 3))
t
(-some 'even? '(1 3 5))
nil
```

If you want to find the last element in the list that matches a predicate function, you need to use the *-last* function as follows:

```
(-last predicate list) ;; Syntax
```

```
(-last 'even? '(1 2 3 4 5))
4
```

The first element of a list can be returned using the *-first-item* function, while the last element of a list can be returned using the *-last-item* function, as shown below:

```
(-first-item list) ;; Syntax
(-last-item list) ;; Syntax
```

```
(-first-item '(1 2 3))
1
```

```
(-last-item '(1 2 3))
3
```

If you wish to return everything but the last item in a list, you can use the *-butlast* function. For example:

```
(-butlast list) ;; Syntax
```

```
(-butlast '(1 2 3 4))
(1 2 3)
```

The *-sort* function sorts the elements of the list based on an input comparison function. An example is shown below:

```
(-sort comparator list) ;; Syntax
```

```
(-sort '< '(4 1 2 3))
(1 2 3 4)
```

The *-list* function takes a list of arguments and returns a new list. For example:

```
(-list arguments) ;; Syntax
```

```
(-list 1 2 3 4)
```

```
(1 2 3 4)
```

Threading macros

The threading macros take an argument *x* and thread the expression through the forms also passed as arguments. A few examples are shown below:

```
(-> x forms) ;; Syntax
--> x forms) ;; Syntax
->> x forms) ;; Syntax
```

```
(-> '(2 3 4) (append '(5 6)))
(2 3 4 5 6)
```

```
(defun square (n) (* n n))
```

```
(->> '(1 2 3) (-map 'square))
(1 4 9)
```

```
--> " small" (concat "a" it " world"))
"a small world"
```

The *-some->*, *-some->>* and *some->>* threading macros evaluate the expression *x* to check whether it is not nil, and then thread it through the first form. If the result is not nil, it continues to the next form, and so on. Examples are provided below for reference:

```
(-some-> x form) ;; Syntax
(-some->> x form) ;; Syntax
(-some--> x form) ;; Syntax
```

```
(-some-> 4 square)
16
```

```
(-some->> '(1 3 5) (-last 'even?) (+ 5)) ;; Define even
function
nil
(-some->> '(1 2 3) (-last 'even?) (+ 5))
7
(-some->> '(1 2 3 4) (-last 'even?) (+ 5))
9
```

```
(-some--> '(1 2 3) (-filter 'even? it) (append it it) (-map
'square it))
(4 4)
```

Binding constructs

The *-when-let* function accepts a variable-value expression, and evaluates it. If the value is non-nil, then it binds it to the variable and executes the body. In the following example, the character *p* is found in index 3, and the match-index gets the value of 3. It is then added to the number 2 to obtain the result